

Tecnológico de Monterrey

Documentación Evidencia 2

Javier Pineda Ponce - A00845219

18/10/2026

Programación de estructuras de datos y algoritmos fundamentales

David Alonso Cantú Delgado

```
PS C:\Users\chapu\C.MTY.TC1031.607.2613.A00845219\Evidences\Evidence2\src\output> & .\'code.exe'

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: 5

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: -9

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: 0

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: 2

No hay datos cargados. Ejecute la Opcion 1 primero.

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
```

```
Opcion: 3
No hay registros cargados.

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: 1

Archivos disponibles:
1. log607-1.txt
2. log607-2.txt
Seleccion: 1
==> Archivo abierto exitosamente en: ../../data/log607-1.txt
Carga finalizada. Registros exitosos: 6818 | Lineas omitidas: 0

Ordenando registros...

--- Resultados del Algoritmo ---
Algoritmo usado: QuickSort
Archivo origen: ../../data/log607-1.txt
Tamanio de los datos: 6818 elementos
Tiempo de ejecucion: 91 ms
Numero de comparaciones: 111859
Profundidad maxima de recursion: 36
Coincidencia con la prediccion: (Complejidad: O(N log N)
---------------------------------
Archivo 'output608.out' generado exitosamente.

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: 2

-- Busqueda de Registros --
```

```
Mes (Ej. Sep): Jun
Dia (Ej. 14): 01
Anio (Ej. 2024): 2025

Se encontraron 7 registros.
Mostrando orden invertido usando Pila:
Jun 1 2025 22:15:34 10.14.108.149 Suspicious IP Addresses
Jun 1 2025 10:14:45 10.14.43.109 Broken authentication and session management
Jun 1 2025 23:55:26 10.14.81.142 Application error
Jun 1 2025 22:42:29 10.14.10.63 Security breaches
Jun 1 2025 22:45:13 10.14.211.77 Social engineering attacks
Jun 1 2025 20:48:45 10.14.106.106 Changes in File Integrity
Jun 1 2025 18:33:12 10.14.37.37 Failed Login Attempts

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: 3

================ ESTADISTICAS DEL REPORTE ================
Total de registros procesados: 6818

Dia con mas incidentes (21 eventos):
 -> Jan 27 2025

IPs mas repetidas (3 apariciones):
 -> 10.14.123.164
 -> 10.14.126.198
 -> 10.14.161.42
 -> 10.14.162.205
 -> 10.14.176.221
 -> 10.14.186.139
 -> 10.14.199.224
 -> 10.14.200.76
 -> 10.14.242.186
 -> 10.14.29.229
```

```
 -> 10.14.36.7
 -> 10.14.69.71
 -> 10.14.94.165

Evento mas frecuente (342 incidencias):
 -> Malware Detection Alerts
============================================================

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: 1

Archivos disponibles:
1. log607-1.txt
2. log607-2.txt
Seleccion: 2
==> Archivo abierto exitosamente en: ../../data/log607-2.txt
Carga finalizada. Registros exitosos: 6818 | Lineas omitidas: 0

Ordenando registros...

--- Resultados del Algoritmo ---
Algoritmo usado: QuickSort
Archivo origen: ../../data/log607-2.txt
Tamanio de los datos: 6818 elementos
Tiempo de ejecucion: 1887 ms
Numero de comparaciones: 2391795
Profundidad maxima de recursion: 700
Coincidencia con la prediccion: (Complejidad: O(N log N)
------------------------------
Archivo 'output608.out' generado exitosamente.

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
```

```
3. Ver Estadisticas Generales
4. Salir
Opcion: 2

-- Busqueda de Registros --
Mes (Ej. Sep): Dec
Dia (Ej. 14): 15
Anio (Ej. 2024): 2023

No hay registros para la fecha solicitada.

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: 2

-- Busqueda de Registros --
Mes (Ej. Sep): Jan
Dia (Ej. 14): 23
Anio (Ej. 2024): 2025

Se encontraron 9 registros.
Mostrando orden invertido usando Pila:
Jan 23 2025 09:36:36 10.14.56.228 Port Scanning Activity
Jan 23 2025 22:55:49 10.14.138.12 Network connectivity issues
Jan 23 2025 16:41:00 10.14.248.91 Social engineering attacks
Jan 23 2025 12:43:46 10.14.236.171 Failed Login Attempts
Jan 23 2025 12:50:52 10.14.8.141 Changes in File Integrity
Jan 23 2025 04:28:07 10.14.100.94 Unencrypted communication
Jan 23 2025 07:28:28 10.14.233.4 Load balancing issues
Jan 23 2025 05:34:42 10.14.84.161 Broken authentication and session management
Jan 23 2025 22:29:54 10.14.172.35 Configuration errors

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
```

```
3. Ver Estadisticas Generales
4. Salir
Opcion: 3

=============== ESTADISTICAS DEL REPORTE ===============
Total de registros procesados: 6818

Dia con mas incidentes (21 eventos):
 -> Jan 27 2025

IPs mas repetidas (3 apariciones):
 -> 10.14.123.164
 -> 10.14.126.198
 -> 10.14.161.42
 -> 10.14.162.205
 -> 10.14.176.221
 -> 10.14.186.139
 -> 10.14.199.224
 -> 10.14.200.76
 -> 10.14.242.186
 -> 10.14.29.229
 -> 10.14.36.7
 -> 10.14.69.71
 -> 10.14.94.165

Evento mas frecuente (342 incidencias):
 -> Malware Detection Alerts
========================================================

--- Analisis de Logs ---
1. Cargar y Ordenar Archivo de Log
2. Buscar registros por fecha (Requiere ordenamiento)
3. Ver Estadisticas Generales
4. Salir
Opcion: 4
Programa finalizado.
```

2. Resultados de ordenamiento

Archivo	**log607-1.txt (Desordenado)**	**log607-2.txt (Casi Ordenado)**
Predicción del usuario	Rápido y eficiente ($O(N \log N)$)	Lento e ineficiente (baja $O(N^2)$)
Tiempo de ejecución (ms)	Menor (aprox. 90 ms)	Mayor (aprox. 111500 ms)
Número de comparaciones	Óptimo ($N\log_2 N$)	Muy elevado ($N^2/2$)
Profundidad máxima de recursión	Baja ($\log_2 N$)	Alta (próxima a N)
¿Conserva el orden de llegada?	No, es inestable	No, es inestable

Con quicksort como vimos en la evidencia anterior es mucho mejor con datos completamente desordenados, ya que dependerá mucho de dónde pongamos el pivote. Al estar muy desordenados cada división del arreglo es muy balanceada y por eso cumple con la expectativa. Por otro lado pasa lo contrario con el archivo casi ordenado, las divisiones tienen un desbalance al estar casi ordenado, y como el pivote no está bien configurado para este archivo específico tiene que hacer más iteraciones para lograr ordenar el archivo.

3. Uso de IA

Asistente: Gemini 3.6 Pensar

Prompt 1: “terminate called after throwing an instance of 'std::bad_alloc' what(): std::bad_alloc salio este error al compilar, explicame y donde tengo que buscar”

Prompt 2: “Tengo muchos errores en mis archivos headers, es raro porque segun ya borre la repetición de node, pero aun asi marca errores a causa del main.cpp”

Me ayudó mucho la IA en este proyecto, diría que mucho más que el anterior. Mis archivos .h los tenía hecho un desastre por cómo los había trabajado durante el periodo, no todos estaban

separados por headers entonces a la primera ocasión empezaron a chocar varias funciones (como el Node). Le tuve que pedir ayuda a la IA porque llegó a un punto que cada vez que movía algo otra cosa dejaba de funcionar, esto me llevó a pedirle a la IA que revisara mis códigos .h y que me sugiriera los cambios necesarios sin cambiar la estructura del código original, al final me sugirió hacer cambios en un par de lineas, me explico el porque y el codigo volvio a funcionar. Y de nuevo le pedí a a la IA que me generara los menos y los cout de forma casi automática, ya que es la parte más tediosa y que puedo verificar rápidamente.

Un error que tuvo la IA y me di cuenta de inmediato era que cuando estaba aún reparando mi archivo Q.h la IA pensaba que aún persistía el mismo problema del node, el cual le dije pero tardo como 3 prompts mas en que aceptara que ya no está el problema del node. Igual la primera vez que le pedí el menú me di cuenta después que al final no era tan bueno como pensaba. Eso me llevo que fuera mas especifico con mi siguiente prompt para generar el menú y esta vez si me lo genero correctamente.

4. Reflexión

El uso de la IA en esta actividad me ayudó a comprender mucho lo importante del orden de los archivos. Sin la IA creo que no hubiera sido capaz de terminar este proyecto a tiempo, ya que estuve mucho tiempo reparando bugs y errores de compilación a causa de mi mala implementación de los headers. Aprendí a hacer un mejor uso de la IA como asistente de desarrollo e inspección de errores de código, ya que no deje que ningún fragmento de código fuera pasado sin previa aprobación e inspección. Un punto especial en donde me ayudó la IA fue con los punteros normales y los inteligentes, que es donde tuve muchos problemas con los tipos de variables.

Lo que yo realice durante este proyecto fueron los headers, la arquitectura y diseño del programa (como la lógica del menú), las definiciones de los tipos de datos y la sobrecarga de operadores para poder ordenar por año, mes y día. Igual verifique el funcionamiento correcto del código y la correcta implementación y búsqueda de los archivos headers y de los archivos de logs, ya que es algo con lo ya había trabajado anteriormente.

La IA me ayudó a identificar muchos errores, un ejemplo fue con el error std::bad_alloc, este es un error de memoria que sucedía por la saturación de esta misma (la causa era un conflicto con el archivo Q.h). Otros errores que me ayudó a solucionar fueron algunas variables mal declaradas que se me pasaban, la mala estructura que tenía en un par de archivos .h y el de los unique_ptr que se soluciono mediante .reset() y .release().
Sin la ayuda de la IA me hubiera tardado mucho más tiempo en este proyecto. Igual hubiera sido muy frustrante el identificar mis errores pasados que me afectaron mucho en el desarrollo de este proyecto (la mala estructura e implementación de los headers). Sin IA me veo desarrollando esto con más dificultad, posiblemente hubiera cambiado archivos enteros o recurrir a mucha ayuda por parte de internet.

Para la estructura del programa termine con la siguiente estructura de datos:

- Vector: se utilizó para ordenar los logs con quicksort porque este necesita tener acceso a cualquier parte del vector para poder mover los elementos del vector y ordenarlos. Si no es un vector básicamente el algoritmo no podría funcionar.
- Queue (Q.h): Este se usa para leer el archivo en orden cronológico. Esto hace que se asegure el orden del archivo original.
- Stack (pilas.h): Me sirvió para hacer la búsqueda por día, aquí se fueron guardando los registros de los logs. Cuando aparecían los depilaba con pop() y quedaban invertidos.

¿Era necesaria la Pila para la opción 3? ¿Qué ventajas y desventajas tiene cada opción?
Realmente no es necesario usar una pila, pero sí tiene algunas ventajas y desventajas. Una de sus mayores ventajas es que actúa como una capa intermedia para almacenar y mover los datos de los logs evitando que se altere el orden. Otra ventaja es que podemos invertir fácilmente la secuencia de manera automática, no hay mucha necesidad de utilizar punteros.
Su principal desventaja es el consumo de memoria, tal como mencioné anteriormente que tenía un problema de memoria fue por esto mismo, un problema con los datos y los nodos hacia que rápidamente utilizara toda la memoria que tenía.

Explicar búsqueda binaria

En una lista ligada sea simple o doble, los nodos están en diferentes partes de la memoria unidos por punteros. Sabiendo esto para acceder a un nodo necesitamos movernos de un punto a otro lo que tiene un costo. Si realizamos un recorrido secuencial de O(N) en cada una de las divisiones del algoritmo, la complejidad iría empeorando, lo que hace que la búsqueda binaria no sea una opción viable, ya que se auto elimina su ventaja principal.

Otras formas de implementar algo similar seria los siguientes métodos:

- Árbol binario: cambiarlo a este tipo de estructura logra un rendimiento de búsqueda O(log N)
- Lista para omitir: Esto reduce el tiempo de búsqueda pero aumenta la complejidad para implementarlo y consume más memoria por puntero.

Con el archivo desordenado quicksort funciona muy bien mientras que el casi ordenado empeora su rendimiento. Algo como el proyecto anterior es que con quicksort funciona mejor cuando está totalmente desordenado por las divisiones, este tarda mucho menos porque cada sección dividida está “balanceada” con la misma cantidad de datos desordenados. Lo que sucede con un archivo casi ordenado es un poco lo contrario, ese balance se rompe y ocupa hacer más iteraciones para ordenar completamente los datos, aparte, con el pivote fijo al final esto hace que con este tipo de datos semi ordenados no funcione como se espera, tener el pivote al final hace que implementar el algoritmo sea sencillo pero tiene este tipo de consecuencias. Lo que pasa con la pila y quicksort es que todo se va empaquetando y apilando en la memoria, como si estuviera recordando lo que hizo.